



ESCUELA SUPERIOR POLITÉCNICA AGROPECUARIA DE MANABÍ
MANUEL FÉLIX LÓPEZ

MAESTRÍA EN DISEÑO WEB Y DESARROLLO DE APPS
II COHORTE PARALELO A

TEMA:

PROYECTO INTEGRADOR DEVOPS - FASE 1

DIAGNÓSTICO Y PLANIFICACIÓN

SISTEMA SELECCIONADO:

SISTEMA WEB CRUD PARA LA GESTIÓN DE USUARIOS

AUTOR:

CRISTHIAN ALFREDO ZAMBRANO ZAMBRANO

MÓDULO: DEVOPS

DOCENTE: MG. ENRIQUE JAVIER MACÍAS ARIAS

AGOSTO DE 2026

01 · CONTEXTO Y FLUJO

Sistema web CRUD para la gestión de usuarios

Aplicación académica que centraliza la creación, consulta, actualización y eliminación de usuarios y la asignación de roles. Beneficia a administradores y personal de soporte que requieren mantener registros consistentes mediante una interfaz web.

Capa	Tecnologías	Responsabilidad
Frontend	React 18, TypeScript, Vite y Mantine	Interfaz, formularios, tabla y consumo de la API.
Backend	Node.js, Fastify, TypeScript y Prisma	Reglas del CRUD, validación y acceso a datos.
Datos	PostgreSQL 17	Persistencia de usuarios.
Ejecución	Docker y Docker Compose	Construcción y coordinación de tres servicios.

Arquitectura actual

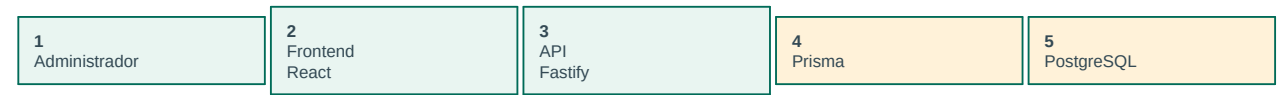


Figura 1. Flujo de solicitudes en la arquitectura actual.

Flujo actual del código al despliegue

No existe CI/CD: una persona modifica, construye, valida y publica. Las esperas no están medidas y la evidencia se limita a la consola y a la comprobación visual.

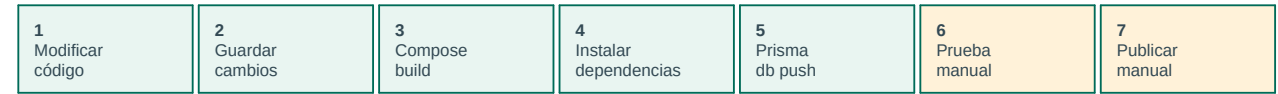


Figura 2. Si falla, se corrige y repite manualmente el flujo.

Etap	Responsable/evidencia	Actividad manual o riesgo
Código/integración	Desarrollador; archivos y commit	Revisión sin validación automática.
Build/datos	Contenedores; consola	Dependencias y db push al arrancar.
Prueba/aprobación	Desarrollador; navegador	CRUD manual, sin suite independiente.
Despliegue/operación	Desarrollador; logs	Sin pipeline, rollback, alertas o métricas.

02 · DIAGNÓSTICO

Cuellos de botella, actividades manuales y riesgos

La prioridad combina impacto y probabilidad de 1 (bajo) a 3 (alto). La puntuación es su producto; los valores 9 se atienden primero.

Hallazgo/tipo	Causa/evidencia	Consecuencia	I×P
Sin CI/CD Proceso/herramienta	Validación mediante comandos locales.	Errores tardíos y baja trazabilidad.	3×3=9
Sin pruebas Proceso	No existen suites unitarias, API o e2e.	Regresiones, retrabajo y aprobación subjetiva.	3×3=9
Dependencias vulnerables Seguridad	npm audit: 2 frontend y 6 altos backend.	Riesgo de cadena de suministro.	3×3=9
Credenciales originales Seguridad	Contraseña en .env/Compose; retirada.	Exposición y acceso no autorizado.	3×2=6
Sin observabilidad Operación	Logs reactivos, sin métricas o alertas.	Detección y diagnóstico tardíos.	2×3=6
Sin rollback/respaldo Continuidad	Sin runbook o restauración probada.	Mayor indisponibilidad o pérdida.	3×2=6

Diagnóstico CALMS

Dimensión	Estado actual	Brecha y respuesta
Cultura	Responsabilidad concentrada.	Compartir decisiones y revisar fallos sin culpa.
Automatización	Compose reproduce servicios.	Automatizar build, pruebas, seguridad y entrega.
Lean	Arquitectura simple.	Reducir lotes y retrabajo con validación temprana.
Medición	Logs, sin línea base.	Recopilar DORA, cobertura e incidentes.
Sharing	Código recibido como ZIP.	Repositorio, README, diagramas y runbook.

Prioridad inmediata	Automatizar compilación y pruebas, bloquear secretos y visibilizar dependencias vulnerables antes de automatizar el despliegue.
----------------------------	---

03 · PROPUESTA INICIAL

Adopción DevOps incremental y medible

Objetivo	Reducir errores manuales y hacer reproducible, segura y medible la entrega mediante validación temprana, configuración protegida y retroalimentación operativa.
----------	---

Alcance y arquitectura preliminar

Dentro del alcance				Fuera del alcance inicial			
GitHub; commits/revisión; CI; pruebas; SAST; dependencias/secretos; imágenes Docker; QA; observabilidad y rollback.				Kubernetes, multirregión y despliegue automático a producción: añadirían complejidad antes de estabilizar el flujo.			
1 Push	2 Build	3 Pruebas	4 Seguridad	5 Imagen	6 QA	7 Desplegar	8 Observar

Figura 3. Flujo propuesto con retroalimentación desde operación.

Trazabilidad problema-práctica-métrica

Problema	Práctica/herramienta	Métrica	Resultado
Validación manual	CI con GitHub Actions o Jenkins.	Lead time/frecuencia	Evidencia automática.
Pruebas inexistentes	Pruebas unitarias y API.	Cobertura/fallos	Menos regresión.
Dependencias/secretos	npm audit, Dependabot, SAST y Secrets.	Hallazgos/versión	Riesgo visible temprano.
Detección tardía	Logs, métricas y OpenTelemetry.	Detección/MTTR	Respuesta más rápida.
Recuperación	Imágenes, rollback, respaldo y runbook.	MTTR/restauración	Continuidad verificable.

Roadmap de 90 días

Periodo	Cultura	Proceso	Técnica/evidencia
0-30	Responsabilidad compartida.	Commits, revisión y criterios.	CI build + secretos; resultado por cambio.
31-60	Revisión sin culpa.	Aceptación y rollback.	Pruebas, SAST y dependencias visibles.
61-90	Revisión de métricas.	Cambios pequeños y QA.	Imágenes, observabilidad y runbook.

04 · MEDICIÓN Y CIERRE

Línea base, metas y gestión de riesgos

No se inventan cifras históricas: la ausencia de medición es parte del diagnóstico. En los primeros 30 días el pipeline y el registro de incidentes generarán una línea base reproducible.

Métrica	Base	Obtención	Meta a 90 días
Lead time	No medido	Commit a despliegue según CI/CD.	Línea base y tendencia descendente.
Frecuencia	No registrada	Despliegues exitosos/mes.	Al menos 2 controlados/mes.
Fallos	No medida	Rollback o corrección / total.	Menor al 20 % tras la base.
Recuperación	No medido	Incidente a restauración.	Runbook y menos de 60 min.
Cobertura	0 % / sin suite	Reporte automático del pipeline.	60 % en lógica crítica.

Riesgos del cambio y controles

Riesgo	Control y evidencia
Seguridad/privacidad	.env excluido, secretos protegidos, escaneo, minimización y retención de logs.
Continuidad	Healthchecks, respaldo/restauración, imágenes versionadas, rollback y runbook.
Adopción	Automatización gradual, guía de contribución y revisión mensual de métricas.

Conclusiones

El sistema es apropiado porque integra interfaz, API y base de datos, pero su flujo original depende de validaciones y despliegues manuales. Los cuellos críticos deben resolverse primero con compilación, pruebas y seguridad tempranas; después, entrega controlada, observabilidad y recuperación. La propuesta conecta cada problema con una práctica, evidencia y métrica, por lo que permite evaluar mejoras sin comparar personas ni inventar datos.

Referencias (APA 7)

DORA. (2024). *Accelerate State of DevOps Report 2024*. Google Cloud.
ESPAM MFL. (2026). *Guía de estudio: Unidad 1 DevOps*.
ESPAM MFL. (2026). *Presentación DevOps, Unidad 1, Sesiones 1-3* [Diapositivas].
Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook* (2nd ed.). IT Revolution.

Repositorio público	https://github.com/cazzSoft/proyecto-integrador-devops-fase1
---------------------	---